PROJECT REPORT

IoT-Based Temperature Monitoring and Overheat Alert System Using Arduino

Submitted By

Akula Ashrith

Department

Electronics and Communication Engineering (ECE)

Task

Task 2 – Sensor-Based Simulation

Date

June 2026

ABSTRACT

The Internet of Things (IoT) has become one of the most significant technological advancements in recent years. IoT enables physical devices to communicate, collect data, and make intelligent decisions with minimal human intervention. Temperature monitoring is a common application of IoT that plays an important role in industries, homes, healthcare facilities, and agricultural environments.

This project presents an IoT-Based Temperature Monitoring and Overheat Alert System using Arduino Uno, TMP36 temperature sensor, LCD display, LED, and buzzer. The system continuously monitors temperature and displays real-time readings on a 16×2 LCD screen. When the temperature exceeds a predefined threshold value of 30°C, the LED and buzzer are activated to alert the user.

The project was successfully designed and simulated using Tinkercad. The implementation demonstrates sensor interfacing, embedded programming, and real-time monitoring concepts used in IoT systems.

TABLE OF CONTENTS

13. Conclusion
14. References

---

1. INTRODUCTION

The Internet of Things (IoT) refers to a network of interconnected devices capable of collecting, transmitting, and processing data through communication networks. IoT technologies are widely used in smart homes, industrial automation, healthcare systems, transportation, and agriculture.

Temperature monitoring is one of the most important applications of IoT. Excessive temperatures can damage equipment, reduce efficiency, and create unsafe working conditions. Automated temperature monitoring systems provide continuous supervision and immediate alerts when abnormal conditions are detected.

In this project, an Arduino Uno is used as the central processing unit. A TMP36 temperature sensor measures environmental temperature, while an LCD display presents the measured values. An LED and buzzer provide warning indications whenever the temperature exceeds the threshold value.

---

2. OBJECTIVES

The main objectives of this project are:

- To monitor temperature continuously.
- To display temperature readings on an LCD screen.
- To generate visual alerts using an LED.
- To generate audio alerts using a buzzer.
- To understand sensor interfacing with Arduino.
- To gain practical knowledge of IoT simulation.
- To implement a basic embedded monitoring system.

---

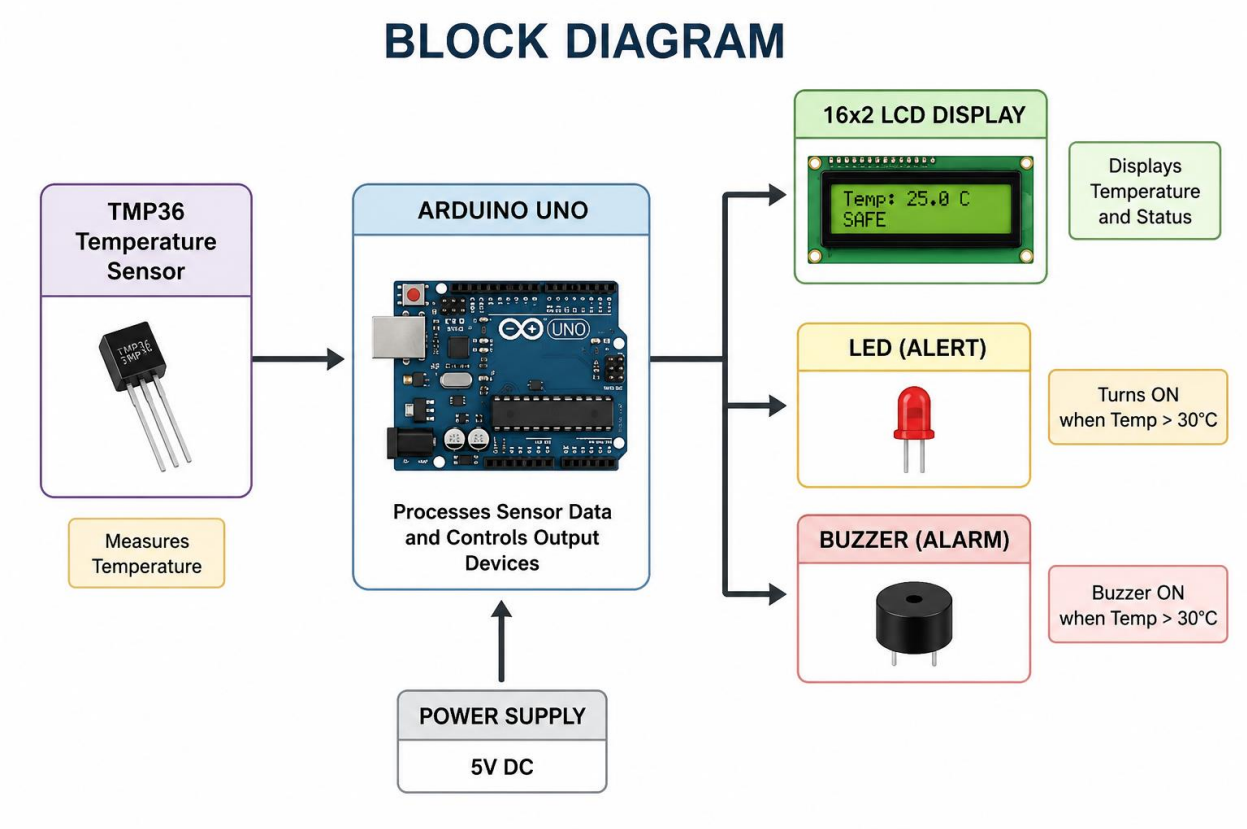
3. COMPONENTS USED

Component	Quantity
Arduino Uno	1
TMP36 Temperature Sensor	1

Component	Quantity
16×2 LCD Display	1
LED	1
Buzzer	1
Breadboard	1
Jumper Wires	Several
220Ω Resistor	2

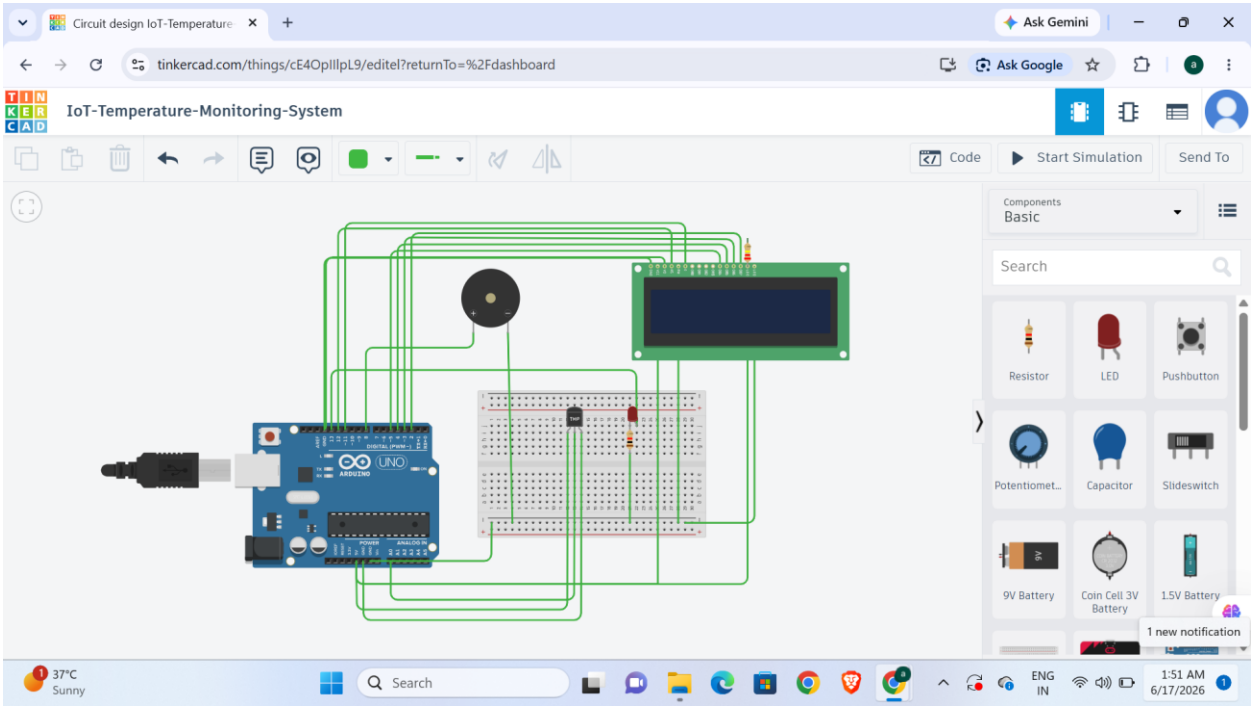
---

4. BLOCK DIAGRAM



The block diagram illustrates the flow of information within the system. The TMP36 sensor measures temperature and sends data to the Arduino Uno. The Arduino processes the sensor data and controls the LCD display, LED, and buzzer based on the measured temperature.

5. CIRCUIT DIAGRAM



The circuit consists of a TMP36 temperature sensor connected to the Arduino's analog input pin. The LCD is interfaced using digital pins for displaying information. The LED and buzzer act as warning indicators whenever the temperature exceeds the threshold value.

6. WORKING PRINCIPLE

The TMP36 temperature sensor continuously senses the ambient temperature and generates an analog voltage corresponding to the measured value.

The Arduino Uno reads this analog voltage through the A0 pin and converts it into temperature values using the built-in Analog-to-Digital Converter (ADC).

The calculated temperature is displayed on the LCD screen. The system continuously compares the measured temperature with a threshold value of 30°C.

When the temperature remains below 30°C:

- LCD displays SAFE status.

- LED remains OFF.
- Buzzer remains OFF.

When the temperature exceeds 30°C:

- LCD displays WARNING HOT.
- LED turns ON.
- Buzzer turns ON.

This provides both visual and audio notifications to the user.

---

7. ARDUINO PROGRAM

```
#include <LiquidCrystal.h>

// LCD Pins: RS, E, D4, D5, D6, D7

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

int sensorPin = A0;

int ledPin = 13;

int buzzerPin = 8;

void setup()

{

  pinMode(ledPin, OUTPUT);

  pinMode(buzzerPin, OUTPUT);

  lcd.begin(16, 2);

  lcd.print("IoT Temp System");

  Serial.begin(9600);

  delay(2000);

  lcd.clear();
```

```
}
void loop()
{
  int sensorValue = analogRead(sensorPin);
  float voltage = sensorValue * (5.0 / 1023.0);
  float temperatureC = (voltage - 0.5) * 100.0;
  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" C");
  lcd.setCursor(0, 0);
  lcd.print("Temp:");
  lcd.print(temperatureC);
  lcd.print((char)223);
  lcd.print("C ");
  lcd.setCursor(0, 1);
  if (temperatureC > 30)
  {
    digitalWrite(ledPin, HIGH);
    digitalWrite(buzzerPin, HIGH);
    lcd.print("WARNING HOT   ");
  }
  else
  {
```

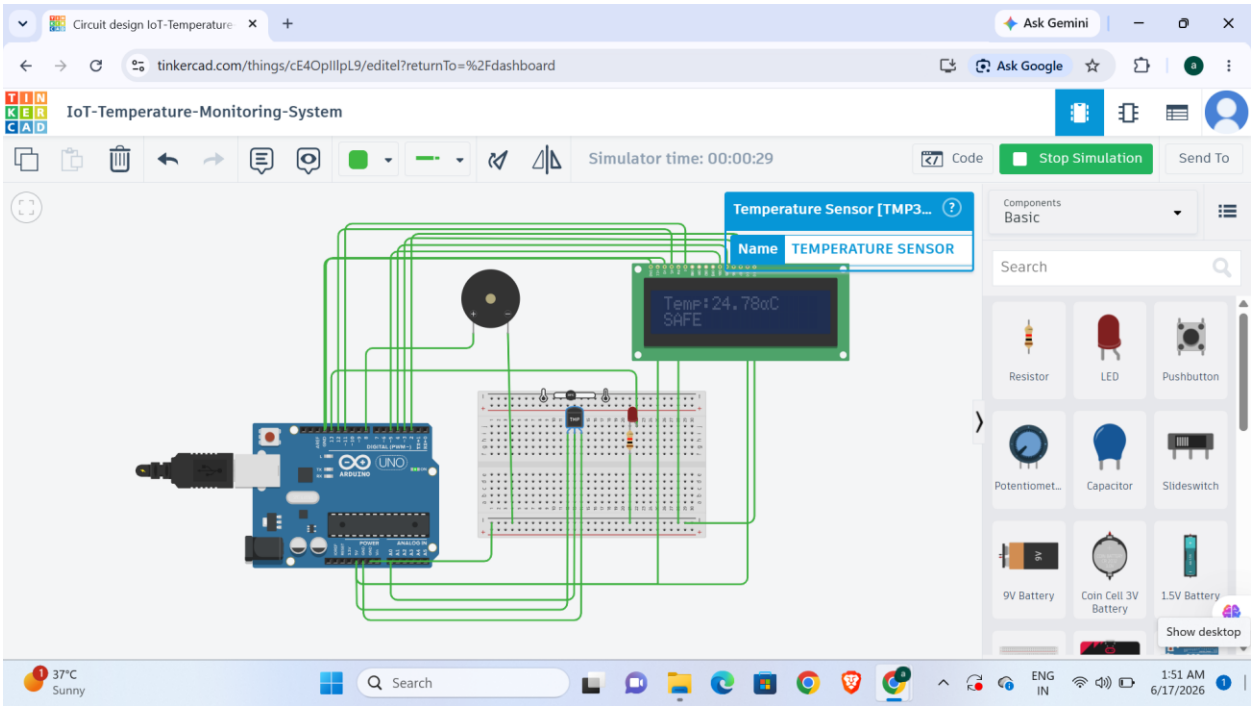
```
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
    lcd.print("SAFE          ");
  }
  delay(1000);
}
```

The Arduino program reads sensor values, converts them into temperature measurements, updates the LCD display, and controls the LED and buzzer according to the temperature condition.

---

8. SIMULATION RESULTS

Case 1: Normal Temperature (25°C)

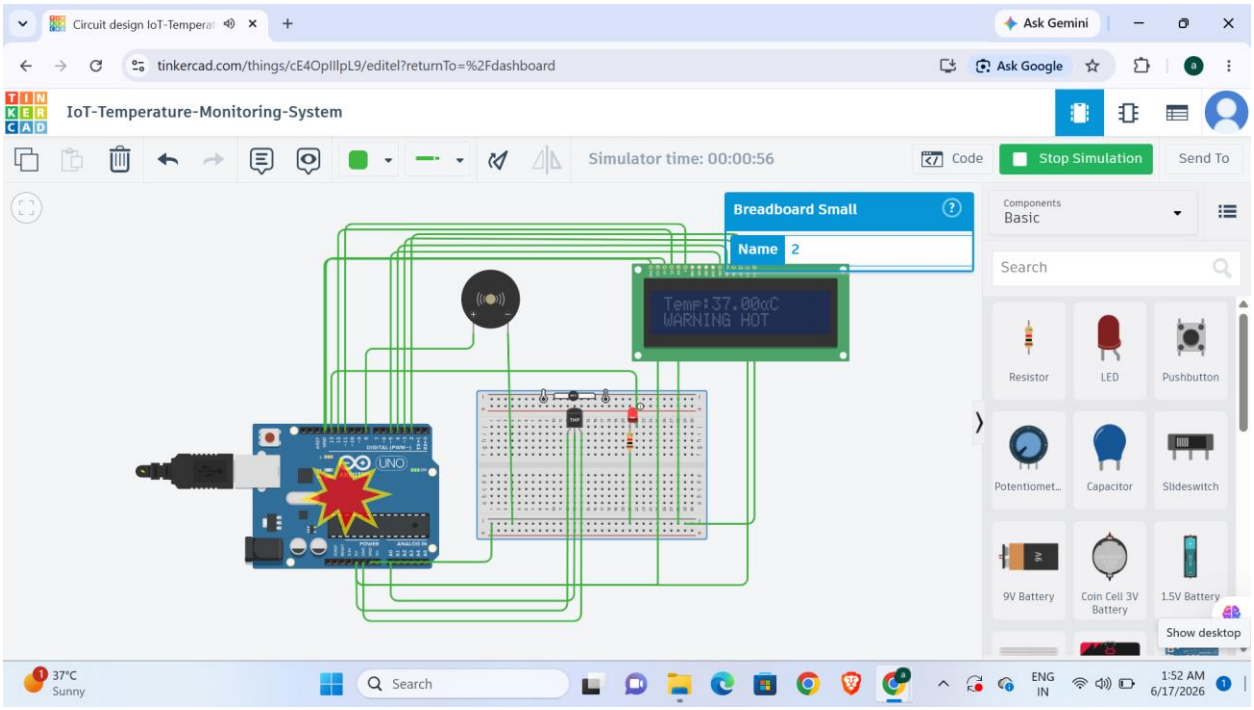


Observations:

- LCD displayed temperature correctly.

- SAFE status was shown.
- LED remained OFF.
- Buzzer remained OFF.

Case 2: High Temperature (35°C)



Observations:

- LCD displayed temperature correctly.
- WARNING HOT message was shown.
- LED turned ON.
- Buzzer turned ON.

Serial Monitor Output

[Insert Screenshot]

The serial monitor successfully displayed real-time temperature values received from the TMP36 sensor.

9. APPLICATIONS

The developed system can be applied in various fields including:

Smart Homes

Used to monitor room temperature and prevent overheating.

Industrial Automation

Used to monitor machine and equipment temperatures.

Agriculture

Used in greenhouses and storage facilities.

Healthcare

Used to monitor storage temperatures for medicines and vaccines.

Server Rooms

Used to protect electronic equipment from excessive heat.

Laboratories

Used for maintaining controlled environmental conditions.

---

10. ADVANTAGES

- Real-time monitoring.
- Low implementation cost.
- Easy to develop and maintain.
- Reliable operation.
- Suitable for educational purposes.
- Expandable for advanced IoT applications.
- Provides immediate warning notifications.

---

11. LIMITATIONS

- No internet connectivity.
- Limited to local monitoring.
- Monitors only one parameter (temperature).
- No data logging capability.

---

12. FUTURE SCOPE

The project can be improved further by incorporating:

- ESP8266 or ESP32 Wi-Fi modules.
- Cloud-based monitoring systems.
- Mobile application integration.
- SMS and email alerts.
- Data storage and analytics.
- Smart home automation features.
- Multiple sensor integration.

These enhancements would transform the project into a complete IoT monitoring solution.

---

13. CONCLUSION

The IoT-Based Temperature Monitoring and Overheat Alert System was successfully designed and simulated using Tinkercad. The system effectively monitored environmental temperature and provided both visual and audio alerts when the temperature exceeded the specified threshold value.

The project helped in understanding IoT concepts, Arduino programming, sensor interfacing, real-time monitoring, and alert generation mechanisms. The successful simulation demonstrates the practical implementation of IoT technologies for safety and monitoring applications.

---

14. REFERENCES

1. Arduino Official Documentation
2. Tinkercad Circuits Documentation

3. TMP36 Temperature Sensor Datasheet
4. CodeAlpha IoT Internship Guidelines
5. Internet of Things Fundamentals and Applications